Task 4 — Terra Incognita

A rover that explores on its own — **BONUS**

Task 4 · bonus · the hard one

https://github.com/KrithinP/vanguard-inductions-2026

Every link in this document opens the live page on GitHub.

PART 1

Your mission

The hard one. Optional, and it can only add

TASK 4 — "Terra Incognita"

The hard one. Optional, and it can only add to your evaluation — but this is the task that tells us who you are as an engineer. **Effort:** 12–18 hours. Do not start this three days before the deadline.

Mission briefing

Your rover can drive, and it can see. It still has to be told where to go.

On Mars nobody tells it. The rover is dropped into terrain no one has mapped, with a 4–24 minute comms delay, and is expected to **work out for itself where it has not been yet, and go there.** That is the whole autonomy problem in one sentence, and by the end of this task you will have solved a version of it.

Three pieces, each one a real capability:

1. **Don't hit things** — a reflex that works from raw laser data with no map at all.
2. **Ask the navigation stack for something** — drive to a pose you choose, in code, and handle it when that fails.
3. **Decide where to go** — look at a half-built map, find the boundary between what you know and what you don't, and send yourself there. Repeatedly, until there is nothing left to explore.

We give you the mapping and navigation infrastructure. You write the autonomy on top of it. That split is deliberate and it is how the real stack works: SLAM and Nav2 are years of other people's engineering, and your job on this team is the layer that decides what to do with them.

How much help you get: less than in earlier tasks. You get the concepts, the message formats, the traps, and the shape of the algorithms. **You do not get working code.** Working out the implementation from a clear description is the skill being tested.

What you're given

`vanguard_navigation/` — a working SLAM and Nav2 configuration, launch files included. Read it, run it, don't rewrite it.

```
cp -r vanguard_navigation ~/vanguard_ws/src/
cd ~/vanguard_ws && colcon build --symlink-install && source install/setup.bash

ros2 launch vanguard_navigation slam.launch.py # mapping
ros2 launch vanguard_navigation nav2.launch.py # navigation
```

`handbook/32-slam-and-nav2.md` explains what it's doing and how to tell when it isn't.

Part A — Sensing · *warm-up*

Add a **2-D LiDAR** to your rover and bridge it. Add a **depth camera** if you want the point cloud; it isn't required.

Then learn RViz properly and **commit a saved `.rviz` config** showing the scan, the robot model and the TF tree at once.

`handbook/30-sensors.md` ·

Part B — A reflex that keeps you alive · *write it yourself*

Write a node that drives the rover forward and **never hits anything**, using `/scan` only. No map, no Nav2, no global knowledge — just the laser and a decision, tens of times a second.

Requirements:

- Subscribe to `sensor_msgs/msg/LaserScan`, publish `geometry_msgs/msg/Twist`.
- Run the control loop on a **timer**, not inside the scan callback.
- Handle `inf` and `NaN` returns correctly. They are not distances and `min(ranges)` on raw data is a bug.
- Divide the scan into sectors and steer toward open space rather than simply stopping. **A rover that stops is not avoiding obstacles, it is refusing to work.**

- Slow down as things get closer instead of driving at full speed until a threshold trips.
- Never drive backwards into something you cannot see — you have no rear sensor.

Prove it: drop the rover in a world with scattered obstacles and let it run for **two minutes without touching anything**. Record it.

`handbook/30-sensors.md` for the message format. The algorithm is yours.

Part C — Commanding the navigation stack · *write it yourself*

Part B has no idea where it is or where it's going. Now use the real stack.

Write a node that sends goals to Nav2's **`NavigateToPose`** action server from code — not by clicking in RViz.

Requirements:

- An `ActionClient` for `nav2_msgs/action/NavigateToPose`.
- **Wait for the server** before sending anything, and say so if it never appears.
- Send a goal, log the **feedback** (distance remaining) as it arrives, and log the **result**.
- Handle all three failure modes distinctly: goal **rejected**, goal **aborted**, and goal **succeeded**. They mean different things and a rover that treats them alike will lie to you.
- Read a list of waypoints from a file, drive them **in sequence**, and report how many were reached out of how many attempted.
- Use the **async** API (`send_goal_async`, `get_result_async`). Blocking calls inside a node deadlock it, and finding that out is part of the task.

Part D — Deciding where to go · *the real task*

Now the rover chooses for itself.

Frontier exploration. A frontier is the boundary between space you know is free and space you have never observed. Drive to frontiers and your map grows; run out of frontiers and you have explored everything reachable.

Write a node that:

1. Subscribes to the SLAM map (`nav_msgs/msg/OccupancyGrid` on `/map`).
2. **Finds the frontier cells** — free cells adjacent to unknown cells.
3. **Groups them into clusters**, because a thousand individual cells is not a thousand places to go. Discard clusters too small to be worth the trip.
4. **Picks one.** Nearest is the obvious choice and it is not the best one — say in your write-up what you chose and why.
5. Converts that cell to a **world pose** and dispatches it using your Part C client.
6. **Repeats** when the goal completes, and **stops cleanly** when no frontiers remain.

Requirements:

- Handle the goal being **aborted** — an unreachable frontier must not deadlock the loop or be retried forever.
- Do not re-send a frontier you have already failed to reach.
- Log each decision: how many frontiers, which you chose, why.
- **Terminate.** A robot that never decides it is finished is not autonomous.

Prove it: start with a blank map and let it explore your arena unattended until it stops. Record it, and save the resulting map.

`handbook/33-occupancy-grids.md` — grid↔world coordinates, and what the cell values mean `handbook/35-exploration.md` — the algorithm, described

---

Part E — Write it up · *short*

`src/task4/NOTES.md`, 300–500 words. Not an essay:

1. **How did you pick which frontier to go to, and what's wrong with your choice?**
2. **What happens when Nav2 aborts a goal?** Describe what your code does and why.
3. **Occupancy grids represent the world as free / occupied / unknown.** Name something real a rover would meet that this **cannot represent**, and what would happen if it drove into one.
4. **What broke, and how did you find it?**

Question 3 is an open problem the team works on. There is no answer key — reasoning honestly beats sounding confident.

Deliverables

`src/task4/`	your avoidance node, action client, and explorer
`src/task4/NOTES.md`	the write-up
`.rviz` config	your saved layout
saved map	the one your rover built by itself
video	under 90 s — the rover exploring unattended, and stopping

In the recording, **say out loud how your code decides where to go next.**

Checks

```
./tools/vanguard check
```

Task 4 never fails your build — it reports only. It is a bonus.

PART 2

Sensing the world

LiDAR, depth and point clouds

30 — LiDAR, depth and point clouds

Time: 45 minutes. **By the end:** your rover senses the shape of the world around it.

Adding a 2-D LiDAR

A laser scanner sweeps a beam around and reports the distance to the first thing it hits at each angle. One horizontal slice of the world, updated many times a second. Cheap, fast, reliable — which is why almost every ground robot has one.

```
<link name="lidar_link">
<visual><geometry><cylinder radius="0.04" length="0.04"/></geometry></visual>
<inertial><mass value="0.2"/>
<inertia ixx="1e-4" ixy="0" ixz="0" iyy="1e-4" iyz="0" izz="1e-4"/></inertial>
</link>

<joint name="lidar_joint" type="fixed">
<parent link="base_link"/>
<child link="lidar_link"/>
<origin xyz="0.15 0 0.12" rpy="0 0 0"/>
</joint>

<gazebo reference="lidar_link">
<sensor name="lidar" type="gpu_lidar">
<update_rate>10</update_rate>
<always_on>true</always_on>
<topic>scan</topic>
<gz_frame_id>lidar_link</gz_frame_id>
<lidar>
<scan><horizontal>
<samples>360</samples>
<resolution>1</resolution>
<min_angle>-3.14159</min_angle>
<max_angle>3.14159</max_angle>
</horizontal></scan>
<range><min>0.15</min><max>12.0</max><resolution>0.01</resolution></range>
</lidar>
</sensor>
</gazebo>
```

Bridge it:

```
/scan@sensor_msgs/msg/LaserScan[gz.msgs.LaserScan
```

`gz_frame_id` matters. It sets the `frame_id` on the outgoing message. If it doesn't name a real link in your TF tree, everything downstream — RViz, SLAM, Nav2 — refuses to use the scan and the error message won't obviously say why.

Look at it

RViz → **Add → LaserScan**, topic `/scan`, Fixed Frame `odom`. Turn *Style* to **Points** and raise *Size* to about 0.05 so you can actually see them.

Drive around. The returns should stay put in the world while the rover moves through them. If they smear or rotate with the robot, your TF is wrong.

```
ros2 topic echo /scan --once | head -20
```

`ranges` is a list of distances, one per angle, starting at `angle_min` and stepping by `angle_increment`. **Out-of-range readings come back as `inf`**, which is not a number — `min(ranges)` on raw data gives you `inf` or garbage. Filter first:

```
valid = [r for r in msg.ranges if msg.range_min < r < msg.range_max]
closest = min(valid) if valid else float('inf')
```

Depth camera and point clouds

Where LiDAR gives one flat slice, a depth camera gives a distance for **every pixel** — a dense 3-D snapshot.

```
<gazebo reference="camera_link">
<sensor name="depth_camera" type="depth_camera">
<update_rate>10</update_rate>
<always_on>true</always_on>
<topic>depth_camera</topic>
<gz_frame_id>camera_link</gz_frame_id>
<camera>
<horizontal_fov>1.047</horizontal_fov>
<image><width>320</width><height>240</height><format>R_FLOAT32</format></image>
<clip><near>0.1</near><far>10.0</far></clip>
</camera>
</sensor>
</gazebo>
```

Bridge the point cloud:

```
/depth_camera/points@sensor_msgs/msg/PointCloud2[gz.msgs.PointCloudPacked
```

> Keep depth at 320×240 and 10 Hz. Point clouds are big, and under software rendering a 640×480 depth camera at 30 Hz will bring the whole simulation to a halt. Nothing here needs the resolution.

Look at it properly

RViz → **Add → PointCloud2**, topic `/depth_camera/points`.

Then spend two minutes on this, because it's worth it:

1. Set **Color Transformer** to `AxisColor` and **Axis** to `Z` — now height is colour, and the ground separates from the obstacles.
2. Switch **Axis** to `X` — now colour is distance from the camera.
3. Rotate the view with the mouse.

There's a moment where it stops looking like coloured noise and starts looking like a room you could walk through. Wait for that moment. It's the difference between knowing what a point cloud is and understanding it.

What you now have

Sensor	Gives you	Costs
LiDAR	one accurate horizontal slice, 360°	blind above and below that slice
Depth camera	dense 3-D, but only where it's pointed	narrow view, noisier, heavy

Neither is sufficient alone, which is why real rovers carry both. Notice also that **both only report surfaces they can see.** Nothing tells you about a hole, an overhang, or what a rock is made of. Hold that thought — the write-up comes back to it.

If it went wrong

No `/scan` topic — the `gz-sim-sensors-system` plugin is missing from the world file, or the bridge isn't running. `gz topic -l` shows Gazebo's side.

Scan appears but RViz shows "No transform" — `gz_frame_id` doesn't match a link in your URDF.

Scan rotates with the robot instead of staying still — Fixed Frame is `base_link`. Set it to `odom`.

Simulation crawls after adding the depth camera — expected. Lower the resolution and rate.

All ranges are `inf` — the LiDAR is inside your chassis, below `range_min`. Move it up or out.

Next: `31-rviz.md`.

PART 3

RViz, properly

The tool you'll use for years

31 — RViz, properly

Time: 45 minutes. **By the end:** RViz stops being a thing that mysteriously shows nothing.

Most people use RViz by trial and error for years. An hour spent learning it properly pays back within a week.

What RViz is, and isn't

RViz is not the simulator. It draws nothing by itself. It subscribes to topics and renders whatever it finds.

That distinction explains most confusion:

- RViz shows nothing → *nothing is publishing*, or you're not subscribed to it.
- RViz shows something wrong → *the data is wrong*. RViz is an honest window.

Closing RViz doesn't stop the robot. Opening it changes nothing. It is a viewer.

Fixed Frame — the setting that breaks everything

Top left, under Global Options. It answers: *what should hold still?*

Everything is drawn relative to the Fixed Frame. Choose one that doesn't exist and **RViz shows nothing at all** and complains about transforms.

Fixed Frame	You see
`base_link`	the world moving around a stationary robot
`odom`	the robot driving through a stationary world
`map`	same, but corrected by SLAM — needs SLAM running

Default is `map`, which doesn't exist until you run SLAM. That single fact is the cause of most "RViz is broken" reports. When in doubt, set it to `odom`.

Displays

Add at the bottom left. The ones that matter here:

Display	Shows	Watch out for
RobotModel	your rover	set *Description Topic* to `/robot_description`
TF	every frame as axes	turn off *Show Names* once it's crowded
LaserScan	LiDAR returns	Style → Points, Size → 0.05
PointCloud2	depth data	Color Transformer → AxisColor
Map	the occupancy grid	topic `/map`
Path	a planned route	one per plan — you'll want two
Costmap	navigation cost	it's a **Map** display on a costmap topic
Image	camera feed	inline, no separate window

Each display has a **Status** line. Green is fine; yellow and red tell you exactly what's missing. **Read it** — it's more informative than most error messages in ROS.

Save your layout

Rebuilding this every time is miserable. **File → Save Config As**, into your package's `rviz/` folder.

Then launch RViz with it already loaded:

```
Node(
 package='rviz2', executable='rviz2',
 arguments=['-d', os.path.join(pkg, 'rviz', 'rover.rviz')],
)
```

A good `.rviz` config is a genuinely useful thing to own, and **committing one is part of this mission's deliverables**.

Interactive tools

Along the top:

- **2D Goal Pose** — click and drag on the map to send Nav2 a destination. Drag sets the final heading.
- **2D Pose Estimate** — tell localisation where the robot really is.
- **Publish Point** — click anywhere, get its coordinates on `/clicked_point`. Surprisingly handy for measuring things.
- **Measure** — distance between two clicks.

Reading a costmap

You'll need this in a moment, so here's the key:

Colour	Meaning
dark / transparent	free space
blue → cyan → red gradient	rising cost near obstacles (**inflation**)
solid black/purple	lethal — a real obstacle
grey	**unknown** — never observed

The coloured halo is wider than the wall on purpose. That's the inflation layer, and working out *why* it's there is one of the write-up questions.

If it went wrong

Completely empty, red errors about transforms — Fixed Frame. Set it to `odom`.

RobotModel display is red — Description Topic isn't `/robot_description`, or `robot_state_publisher` isn't running.

Laser scan visible but the robot isn't — RobotModel display not added, or the URDF isn't being published.

Everything flickers or jumps — two nodes publishing the same transform, or `use_sim_time` set on some nodes and not others.

RViz crashes on startup — graphics. Common in Docker and VMs:

```
export LIBGL_ALWAYS_SOFTWARE=1
rviz2
```

Next: `32-slam-and-nav2.md`.

PART 4

What you're given

SLAM and Nav2 - run them, don't rewrite them

32 — SLAM, Nav2 and costmaps

> **Time:** 90 minutes. **By the end:** your rover has built a map and driven itself across it, and you understand the stack you're about to build on top of.

This page is about the part you don't write. SLAM and Nav2 are each years of work by large teams; the configuration is provided in `vanguard_navigation/`. Read it, run it, learn to tell when it's misbehaving — then leave it alone.

Everything else in Task 4 you write yourself: the reflex that keeps the rover alive, the client that commands Nav2, and the logic that decides where to go. This page is the foundation those sit on, and you will debug them far faster if you know what a healthy stack looks like first.

Getting the provided package

```
cp -r vanguard_navigation ~/vanguard_ws/src/
cd ~/vanguard_ws && colcon build --symlink-install && source install/setup.bash
```

Docker users: already installed — nothing to do.

Native users:

```
sudo apt install -y ros-jazzy-slam-toolbox ros-jazzy-navigation2 ros-jazzy-nav2-bringup
```

SLAM

Simultaneous **L**ocalisation **A**nd **M**apping. The name states the circularity: to build a map you must know where you are, and to know where you are you need a map. SLAM solves both at once, each half constraining the other.

Roughly what happens each scan:

1. Guess where you moved, using odometry.
2. **Scan matching** — slide the new scan against the map until it lines up best. That correction is better than the odometry guess.
3. Write the scan into the map at the corrected position.
4. Occasionally recognise somewhere you've been before (**loop closure**) and correct all the accumulated drift at once.

Step 4 is why a mapped corridor comes out straight instead of gently curving away.

Run it

```
# Terminal 1 — simulator, rover spawned, bridge running (as before)
# Terminal 2
ros2 launch vanguard_navigation slam.launch.py
# Terminal 3
rviz2
```

In RViz: Fixed Frame `map`, then **Add → Map**, topic `/map`.

Now drive slowly with `teleop_twist_keyboard` and watch. Grey becomes black (occupied) and white (free) as the LiDAR sweeps.

> **Drive slowly.** Scan matching needs consecutive scans to overlap. Spinning fast is the most reliable way to produce a smeared, useless map — and, if you want, a good thing to try deliberately once.

Watch the transforms:

```
ros2 run tf2_ros tf2_echo map odom
```

That's SLAM's output: the correction between where odometry thinks you are and where you actually are. It starts near zero and grows as odometry drifts.

Save the map

```
ros2 run nav2_map_server map_saver_cli -f ~/vanguard_ws/arena_map
```

Two files: `arena_map.pgm` (an image — open it, it's just a picture) and `arena_map.yaml` (resolution, origin, thresholds). **Commit both.**

Occupancy grids

The map is a grid of cells, each holding one number:

Value	Meaning
`0`	free
`100`	occupied
`-1`	**unknown**

That third value carries more weight than it looks. "I have never seen this" is not "this is clear". A planner that treats unknown as free will happily route through a wall it hasn't looked at yet.

Look at `allow_unknown: true` in `nav2_params.yaml` and think about what it's buying and what it's risking.

Nav2

Nav2 is not one algorithm. It's a set of servers coordinated by a **behaviour tree**:

Server	Job
planner	route from here to the goal, over the global costmap
controller	velocity commands to follow that route, over the local costmap
behaviour	what to do when stuck — spin, back up, wait
bt_navigator	runs the tree that decides the order of all of it

The tree is what makes Nav2 feel intelligent: *plan; follow; if the controller fails, clear the costmap and retry; if that fails, back up and replan.*

Lifecycle nodes — the thing that will confuse you

slam_toolbox and every Nav2 server are *lifecycle* nodes. They boot into an `unconfigured` state where they subscribe to nothing, publish nothing, and log nothing. Something has to walk them through *configure* → *activate* before they do any work.

That is why both provided launch files include a `lifecycle_manager`. Without one, `ros2 node list` shows the node, `ps` shows the process, and **nothing whatsoever happens** — no error, no warning, no map.

Check the state directly:

```
ros2 service call /slam_toolbox/get_state lifecycle_msgs/srv/GetState "{}"
```

```
response: State(id=3, label='active')
```

`label='unconfigured'` means it never started properly. `label='active'` means it's working. Another quick tell:

```
ros2 topic info /scan
```

An active SLAM node appears as `Subscription count: 1`. A count of **0** means nothing is listening to your laser, however healthy everything looks.

If anything in this mission silently does nothing, check lifecycle state first. It is the single most likely cause, and it produces no error message at the point you notice the problem.

Run it

With the simulator and SLAM already running:

```
ros2 launch vanguard_navigation nav2.launch.py
```

Wait for the lifecycle manager to report everything active. Then in RViz click **2D Goal Pose** and drag somewhere on the map.

The rover should plan a route and drive it.

The costmaps — the part to actually study

Add two **Map** displays in RViz:

- topic `/global_costmap/costmap`
- topic `/local_costmap/costmap`

and two **Path** displays:

- `/plan` — the global route
- `/local_plan` — the controller's short-term trajectory

Global vs local

	Global	Local
Covers	the whole known map	a 4×4 m window
Frame	`map`	`odom`
Moves with the robot?	no	**yes** (`rolling_window: true`)
Updated	~1 Hz	~5 Hz
Built from	the saved map + sensors	live sensors only
Used by	the planner	the controller

The split exists because the two jobs have opposite needs. Planning a route across a building needs breadth and can be slow. Not hitting the chair in front of you needs speed and only needs to see a few metres. One costmap serving both would be too slow, too big, or both.

Note the frames too: the local costmap lives in `odom` because the controller needs **smooth** data — a `map`-frame jump from loop closure would make the rover twitch.

Inflation

Turn on the costmaps and look at the coloured halo around every obstacle. It's wider than the obstacle.

That's the **inflation layer**, and it's doing something clever: it grows every obstacle by the robot's radius, so the planner can pretend the robot is a dimensionless **point**. Checking "does this point collide" is enormously cheaper than checking "does this 0.6 × 0.4 m rectangle collide, at this orientation" — and inflation makes them equivalent.

`inflation_radius: 0.55` with `robot_radius: 0.35` leaves ~0.2 m of margin.

Try changing it. Set `inflation_radius` to `0.1`, rebuild, and watch the rover cut corners and clip obstacles. Set it to `2.0` and watch it refuse to enter a doorway it fits through easily. That experiment will teach you more than this page.

Things to break on purpose

Do this **now**, before you write any of Task 4. Every one of these will happen to you accidentally later, and recognising it in five seconds instead of an hour is the entire return on this page:

- Put an obstacle in front of the rover **while** it's driving. Watch the local costmap update and the local plan bend around it.
- Set a goal inside a wall.
- Set a goal in grey unknown space.
- Drop `inflation_radius` to 0.1 and drive through a tight gap.
- Spin the rover fast during mapping, then try to navigate the resulting map.
- Drive somewhere, then physically teleport the model in Gazebo. Watch odometry and the map disagree.

If it went wrong

Map never appears, and slam_toolbox prints nothing at all — it is almost certainly stuck in `unconfigured`. Check with the `get_state` call above. If you launched the node by hand with `ros2 run` instead of using `ros2 launch vanguard_navigation slam.launch.py`, there is no lifecycle manager and it will never activate. Use the launch file.

Map never appears but SLAM is `active` — Fixed Frame isn't `map`, or it isn't getting `/scan`. Check `ros2 topic hz /scan` and `ros2 topic info /scan`.

`maximum laser range setting (25.0 m) exceeds the capabilities of the used Lidar` — harmless, but it means `max_laser_range` in the config doesn't match your sensor's `<range><max>`. Make them agree.

Map appears then smears into nonsense — driving too fast, or odometry is bad.

"2D Goal Pose" does nothing — Nav2 servers didn't activate. Read the lifecycle manager log.

Rover plans a path but doesn't move — check whether anything reaches `/cmd_vel` (`ros2 topic echo /cmd_vel`). If the plan exists but no velocities flow, the controller failed to activate. **Also check the message type.** On our

Jazzy stack (Nav2 1.3.12) `/cmd_vel` is `geometry_msgs/msg/Twist`, which matches the bridge line from Task 2 — but newer Nav2 releases switched to `TwistStamped`, and if the two ends disagree the messages vanish with no error at all. Confirm with:

```
ros2 topic info /cmd_vel -v
```

Rover spins in place forever — a recovery behaviour is running because the controller thinks it's stuck. Usually a bad costmap or a goal it can't reach.

"Timed out waiting for transform map to base_link" — SLAM isn't publishing, or `use_sim_time` is inconsistent. It must be `true` everywhere when using Gazebo.

Everything is unbearably slow — run Gazebo headless (`gz sim -s`) and watch in RViz only.

Back to the Task 4 sheet.

PART 5

Reading a map in code

Occupancy grids, and grid-to-world

33 — Occupancy grids

By the end: you can read a map in code, and convert between grid cells and places in the world.

The message

```
ros2 interface show nav_msgs/msg/OccupancyGrid
```

```
std_msgs/Header header
MapMetaData info
 float32 resolution # metres per cell
 uint32 width # cells across
 uint32 height # cells up
 geometry_msgs/Pose origin # world pose of cell (0,0)
int8[] data # width*height values, ROW-MAJOR
```

`data` is **one flat list**, not a 2-D array. The cell at column `x`, row `y` is:

```
index = y * width + x
```

Get that backwards and your map is transposed — which looks plausible for a square arena and wrong for everything else. Sanity-check it early.

The three values

Value	Meaning
`0`	free — observed, nothing there
`100`	occupied — observed, something there
`-1`	**unknown** — never observed

Values in between exist in some maps and mean "probably". slam_toolbox mostly gives you the three above.

`-1` is the one this task is built on. "I have never looked here" is not "it is clear", and the boundary between `0` and `-1` is exactly what you are hunting.

Grid ↔ world

Every cell has a real place in the world. You need this in both directions: frontiers are found in cells, but Nav2 wants a pose in metres.

```
world_x = origin.x + (col + 0.5) * resolution
world_y = origin.y + (row + 0.5) * resolution
```

and back:

```
col = floor((world_x - origin.x) / resolution)
row = floor((world_y - origin.y) / resolution)
```

Three things that catch people:

- **The `+ 0.5`.** Without it you get the cell's corner, not its centre. Half a cell of error, every time, in the same direction.
- **`origin` is usually negative**, because SLAM puts the robot near the middle and grows the map outward. If you assume the map starts at (0,0) everything lands in the wrong place.
- **The map's origin moves.** As slam_toolbox expands the map, `origin` changes between messages. Cache a cell index from an old map and it now means somewhere else. **Work from the message you were just handed**, not from remembered indices.

Reading it efficiently

The map is tens of thousands of cells and arrives several times a second. A double `for` loop in Python over every cell, every message, will not keep up.

```
import numpy as np
grid = np.array(msg.data, dtype=np.int8).reshape(msg.info.height, msg.info.width)
```

Now you have a proper 2-D array and can ask numpy questions about all of it at once — `grid == 0`, `grid == -1`, and so on. Neighbour tests become array shifts rather than nested loops. Work out how to express "free cell next to an unknown cell" that way; it is the difference between a node that runs at 5 Hz and one that stalls the whole system.

> **The QoS trap.** `/map` is published **transient local** (latched) — it is sent once and held for late subscribers. Subscribe with default QoS and **you will receive nothing at all**, with no error. You need: `python from rclpy.qos import QoSProfile, QoSDurabilityPolicy, QoSReliabilityPolicy qos = QoSProfile(depth=1, durability=QoSDurabilityPolicy.TRANSIENT_LOCAL, reliability=QoSReliabilityPolicy.RELIABLE)` This costs people hours. From the command line the same thing applies: `ros2 topic echo /map --once --qos-durability transient_local`

Look at it before you trust it

Save a map and open the `.pgm` in any image viewer. White is free, black is occupied, grey is unknown. If your code disagrees with the picture, the code is wrong.

Next: `34-action-clients.md`.

PART 6

Commanding Nav2

Actions, async, and handling failure

34 — Actions, and commanding Nav2

> **By the end:** you can drive the navigation stack from code, and handle it properly when it fails.

Why actions exist

You've used **topics** — fire and forget, no reply. Some things need more than that. "Drive to this pose" takes a minute, you want progress while it happens, you might want to cancel, and at the end you need to know whether it worked.

That's an **action**. Three parts:

Goal	what you want. Sent once.
Feedback	progress, streamed while it runs
Result	how it ended, once

Nav2's is `nav2_msgs/action/NavigateToPose`:

```
ros2 interface show nav2_msgs/action/NavigateToPose
ros2 action list # /navigate_to_pose should be there once Nav2 is up
```

Try it by hand first, before writing anything:

```
ros2 action send_goal /navigate_to_pose nav2_msgs/action/NavigateToPose \
 "{pose: {header: {frame_id: map}, pose: {position: {x: 1.0, y: 0.0}, orientation: {w: 1.0}}}}" --feedback
```

If that doesn't move the rover, your code won't either. Fix it here.

The shape of a client

You need, in order:

1. An `ActionClient(self, NavigateToPose, '/navigate_to_pose')`.
2. **`wait_for_server()` with a timeout.** If Nav2 isn't up, say so and stop — don't hang forever with no output. This is the single most common way a beginner's node appears to "do nothing".
3. Build a `NavigateToPose.Goal()`. The pose needs a **`frame_id`** (`map`) and a **valid orientation quaternion** — all zeros is not a rotation and Nav2 will reject or misbehave. `w = 1.0` is "facing along +x".
4. `send_goal_async(goal, feedback_callback=...)`. This returns a **future**, not a goal handle.
5. When that future resolves, check `goal_handle.accepted`. **A rejected goal never produces a result** — if you wait for one anyway, you wait forever.
6. `goal_handle.get_result_async()` for another future. When *that* resolves, read `result.status` and compare against `GoalStatus` (`STATUS_SUCCEEDED`, `STATUS_ABORTED`, `STATUS_CANCELED`).

Async, and why you cannot avoid it

The tempting version is:

```
future = client.send_goal_async(goal)
rclpy.spin_until_future_complete(self, future) # inside a node method
```

Do not do this inside a callback. `spin` while already spinning **deadlocks** — the node stops processing the very messages the future is waiting on. It looks like a hang with no error, and it is the classic ROS 2 beginner deadlock.

Chain callbacks instead:

```
send_goal_async(...) -> .add_done_callback(self.on_goal_response)
 on_goal_response -> check accepted, then get_result_async()
 .add_done_callback(self.on_result)
 on_result -> read the status, decide what to do next
```

Each step hands control back to ROS. Nothing blocks. Your explorer's "pick the next frontier" logic lives at the end of `on_result` — which is also why the whole thing naturally becomes a loop.

Failure modes, and why they differ

Outcome	Meaning	Reasonable response
Rejected	Nav2 refused it — often a malformed pose or an inactive server	Fix the goal; retrying identically won't help
Aborted	It tried and gave up — unreachable, or stuck	Give up on **this** goal, pick another
Canceled	You cancelled it	Whatever you cancelled it for
Succeeded	It arrived	Carry on

Treating aborted as succeeded is the bug that makes an explorer loop forever on a frontier it cannot reach. Keep a record of goals that failed and don't offer them again.

Feedback

Your callback receives progress — including `distance_remaining`. Useful for logging, and for noticing a rover that has stopped making progress while Nav2 still thinks it's working.

Don't do heavy work in there. It fires often.

Diagnosing

Symptom	Look at
Nothing happens, no output	Did `wait_for_server` succeed? Is `ros2 action list` showing it?
Node hangs silently	A blocking `spin_until_future_complete` inside a callback
Goal instantly rejected	Missing `frame_id`, or an all-zero quaternion
Always aborts	Goal in unknown or occupied space — check against the map
Rover moves, code never notices	You never subscribed to the result future

Next: `35-exploration.md`.

PART 7

Deciding where to go

Frontier exploration - the algorithm, described

35 — Frontier exploration

By the end: you know the algorithm well enough to implement it. This page deliberately contains **no working code** — describing an algorithm clearly and then building it is the skill this task is testing.

The idea

A **frontier** is the boundary between what you know and what you don't: a cell you have observed to be **free**, that touches a cell you have **never observed**.

Stand on a frontier and you can see into the unknown. So:

> Drive to a frontier. The map grows. Compute frontiers again. Repeat. When there are none left, you have explored everything reachable — and you are done.

That's the whole algorithm. Everything below is making it work.

The loop

```
1. take the latest map
2. find frontier cells
3. group them into clusters
4. throw away clusters that are too small
5. score the survivors, pick one
6. convert its centre to a world pose
7. send it to Nav2 and wait for the result
8. succeeded or aborted -> back to 1
 no clusters left -> stop, and say so
```

Step 2 — finding frontier cells

A cell is a frontier if it is **free** (`0`) **and** at least one of its neighbours is **unknown** (`-1`).

Choose 4-neighbour or 8-neighbour and be consistent. 8 gives thicker, better connected frontiers; 4 gives cleaner ones. Either is defensible.

Do this with array operations, not nested loops. The trick: build a boolean array of "is unknown", shift it by one cell in each direction, OR the shifts together to get "has an unknown neighbour", and AND that with "is free". A few lines of numpy, and fast enough to run every time the map updates.

Step 3 — clustering

You now have thousands of individual cells. They are not thousands of destinations — they're a handful of *regions*, each worth one trip.

Group cells that touch into one cluster. A flood fill or breadth-first search over the frontier mask is enough; you don't need a library. For each cluster keep its **size** and its **centroid**.

Step 4 — throwing things away

Small clusters are usually noise — a stray unknown cell behind a table leg, or sensor jitter at the edge of the laser's range. Driving to them wastes the whole run.

Set a minimum cluster size. Somewhere around **10–20 cells** is sensible for a 0.05 m map; tune it and say what you picked. **Too small and your rover chases ghosts forever; too large and it stops with real territory unexplored.** Both failures are worth seeing at least once.

Step 5 — choosing

The obvious rule is **nearest first**. It's simple, it works, and it is not very good — the rover ping-pongs across the arena as new frontiers open behind it.

Better scores balance competing things:

- **distance** — near is cheap

- **size** — a big frontier reveals more map per trip
- **direction** — one already ahead of you avoids a turn
- **stickiness** — continuing to explore the region you're in beats crossing the map for a marginally closer cell

You are not required to do anything clever. **You are required to say what you did and what's wrong with it.** An honest "I used nearest-first and it ping-pongs between two corridors" is worth more than a complicated score you can't explain.

Step 6 — cell to pose

Use the conversion in `33-occupancy-grids.md`, and remember the half-cell offset.

A frontier cell is on the boundary of the unknown, which is often a poor place to stand. Nav2 may refuse it, or abort trying. Two common repairs: aim at a free cell a little way *back* from the frontier, or check the goal has some clearance from occupied cells before sending it. Both are fair game.

Orientation: face the unknown. Or don't and set `w = 1.0` — but a rover that arrives facing the wall it just mapped sees nothing new, which is worth thinking about.

Step 7 — the failure that will bite you

A frontier your rover cannot reach will be regenerated every single cycle, because failing to get there doesn't observe it, so it stays a frontier forever.

Nearest-first plus no memory equals an infinite loop on the closest unreachable cell. Your rover will look busy and achieve nothing, and this *will* happen to you.

Keep a **blacklist** of failed goals and skip anything near one. Decide how near, and whether entries ever expire — a frontier unreachable now may open up later.

Step 8 — knowing when to stop

No clusters above your minimum size means done. **Say so**: log it, publish a final zero velocity, save the map. A robot that quietly keeps spinning has not finished, it has failed silently.

Guard against the other end too: cap total runtime or total goals, so a bug can't leave it running all night.

Watching it work

Publish your frontiers as `visualization_msgs/msg/MarkerArray` and add a **Marker** display in RViz. Seeing clusters appear, get chosen, and vanish as they're explored turns this from guesswork into something you can actually debug.

Optional, and the single highest-value hour you can spend on this task.

When it goes wrong

Symptom	Usually
No frontiers found, ever	Map QoS — you're receiving nothing. See 33
Frontiers everywhere, including inside walls	Free/unknown test inverted, or row/column swapped
Rover drives to the same place forever	Unreachable frontier, no blacklist
Goals always rejected	Pose in unknown space, bad quaternion, or wrong `frame_id`
Explorer stops immediately	Minimum cluster size too high, or map not arrived yet
Everything stalls after one goal	Blocking call in a callback — see 34
Node can't keep up	Nested Python loops over the grid. Use numpy

---

Back to the Task 4 sheet.